

DiaryKG: Real-Time Semantic Corpus Ingestion via Offline Pre-Computation

Technical Disclosure Document

Authors: Eric G. Suchanek, PhD — Flux-Frontiers **Date:** 2026-03-26 **License:** Elastic 2.0 **Repository:** https://github.com/Flux-Frontiers/diary_kg **Status:** Working implementation; validated on full 9-year corpus, consumer hardware

1. The Problem

1.1 The Standard Assumption Is Wrong

The standard industry assumption is that **semantic understanding requires live inference**. When a user asks a question of a document corpus, an LLM processes the query at query time, using a GPU farm or a cloud API. Every question costs money and adds latency. This model scales linearly with query volume and cannot operate offline or on consumer hardware for large corpora.

A second assumption, equally widespread: **temporal grounding requires LLM extraction**. Dates and time references embedded in prose must be parsed by a language model before they can be used for chronological reasoning. For small (4B-parameter) models, this fails entirely — they hallucinate dates, confuse formats, or produce malformed output. This blocks local execution of any temporally-aware memory system.

This document describes a system that breaks both assumptions.

1.2 What Was Missing

No existing open-source pipeline provides all of the following in a single, configurable, CPU-only tool that completes on a consumer laptop:

1. Semantically enriched chunking with topic and context classification
 2. Parallel multi-process vector embedding with no external service dependency
 3. Structured knowledge graph (structural + vector indices) with temporal provenance on every node
 4. Navigable temporal axis in embedding space as a first-class geometric primitive
 5. Sub-3-minute full-corpus ingestion at the scale of 6,000–9,000 entries
-

2. The Solution: Offline Semantic Pre-Computation

The core insight is a shift from **query-time inference** to **ingest-time pre-computation**.

Semantic understanding — topic labeling, context classification, embedding — happens once, when the corpus is ingested. The result is a frozen, versioned, queryable

artifact. At query time, the system performs only vector lookup and graph traversal — both are sub-millisecond operations with no model loading or inference required.

This inverts the cost model:

Model	Ingest cost	Query cost	Privacy	Offline
Cloud LLM	Low	High (per-token)	No	No
Local LLM (query-time)	Low	High (GPU/CPU)	Yes	Yes
DiaryKG (pre-compute)	Medium (once)	Negligible	Yes	Yes

The pre-computation cost is paid once per corpus. Every subsequent query is free.

3. System Architecture

3.1 Corpus Format

The pipeline operates on a pipe-delimited enriched text format, one entry per line:

```
TIMESTAMP | TYPE | CATEGORY | CONTENT
```

Every entry carries its own temporal anchor (TIMESTAMP), semantic type label (TYPE), and context category (CATEGORY). This format is the output of a preprocessing layer that enriches raw diary or journal text with spaCy NLP — named-entity recognition, sentence segmentation, and domain-specific topic classification via a configurable YAML vocabulary. The enriched format is the canonical input to all downstream pipeline stages.

Validated corpus: Samuel Pepys’ diary (1660–1669), 6,450 entries, 23,235 enriched lines, 3.3 MB.

3.2 Pipeline Stages

The pipeline has six sequential stages. Stages 1–4 produce the knowledge graph. Stages 5–6 produce the embedding cache used for manifold analysis and temporal flight.

```
Stage 1 → DiaryTransformer (NLP enrichment + chunking)
Stage 2 → Corpus Emission (.md chunks with YAML frontmatter)
Stage 3 → DockKG SQLite build (structural graph, BM25)
Stage 4 → DockKG LanceDB build (semantic vector index)
Stage 5 → Multi-process Embedder (parallel sentence-transformer)
Stage 6 → JSON Cache (embeddings + texts + timestamps)
```

Stage 1: DiaryTransformer — NLP Enrichment and Chunking The DiaryTransformer class applies five NLP phases to the enriched source text:

1. **Parse:** Read `TIMESTAMP | TYPE | CATEGORY | CONTENT` lines; validate ISO timestamps.
2. **Chunk:** Apply sentence-group chunking (default: 4 sentences/chunk, 512 char max). Hybrid and semantic strategies also available.
3. **Topic classify:** Multi-label classification using a configurable YAML vocabulary. Each chunk can carry multiple topic labels (e.g., `pepys_domestic`, `pepys_court`, `work`).
4. **Context classify:** Single-label context assignment (Office, Home, Social, Reflection, etc.) from a second YAML vocabulary.
5. **Sample with diversity:** When a batch size is specified, temporally-diverse sampling ensures representative coverage across the full time span, not front-loaded.

Parallelism: workers parameter distributes feature extraction across CPU cores via multiprocessing.Pool. Each worker processes an independent shard with no shared state.

Stage 2: Corpus Emission Each chunk is written as a Markdown file under `.diarykg/corpus/` with YAML frontmatter:

```
---
source_file: pepys/pepys_enriched_full.txt
timestamp: 1666-09-02T08:00
topic: pepys_domestic
context: Home
chunk_index: 0
---
```

Temporal provenance is written directly to frontmatter at this stage — not extracted by any LLM. This bypasses the small-model temporal extraction failure mode entirely. Every chunk knows exactly when it happened.

Stage 3 & 4: DockKG Build DiaryKG delegates to DockKG to index the corpus: - **SQLite index** (structural graph): BM25 lexical search, metadata filtering, graph edges (CONTAINS, SIMILAR, TEMPORAL) - **LanceDB index** (vector graph): 768-dim embeddings from `all-mpnet-base-v2`, cosine ANN search

Both indices are populated from the `.md` chunk files in a single `dockg build` pass. The result is a hybrid search system: BM25 for exact term matching, vector search for semantic similarity.

Stage 5: Multi-Process Embedder `pepys_embedder.py` runs independently of the KG build, producing an embedding cache optimized for manifold analysis:

Parse → (optional temporal sample) → shard across workers → each worker:
load SentenceTransformer locally → encode shard → return float32 array
→ concatenate shards → write JSON cache

Key design decisions: - Each worker loads its own model instance. No shared state, no GIL contention. - spawn start method enforced (POSIX systems) for clean subprocess isolation. - Temporal sampling via uniform stride across the time axis — not random sampling — ensures manifold coverage of the full 9-year span. - Output: {embeddings: [...]], texts: [...], timestamps: [...]} — aligned arrays, self-contained.

Stage 6: JSON Embedding Cache The cache is consumed by the manifold analysis and temporal flight components. Format is intentionally simple: three aligned JSON arrays. No database dependency. Portable across environments.

4. The Knowledge Graph: Measured State

Build date: 2026-03-27T01:21:25 UTC **Hardware:** Apple Silicon laptop (consumer hardware, no GPU) **Build time:** ~3 minutes (full corpus, cold start)

4.1 Corpus Metrics

Metric	Value
Source file	pepys_enriched_full.txt
Raw source size	3.3 MB
Diary entries	6,450
Semantic chunks	6,647
Temporal span	1660-01-01 → 1669-08-02
Years covered	9.6 years

4.2 Knowledge Graph Scale

Metric	Value
KG nodes	29,402
KG edges	355,250
SQLite DB size	109 MB
LanceDB vector index	102 MB
Total KG footprint	241 MB

4.3 Topic Distribution (top 10 of 30+ labels)

Topic	Chunks
pepys_domestic	2,297
pepys_court	1,793
work	884
pepys_social	294

Topic	Chunks
pepys_locations	276
did	242
pepys_religious	119
social	118
pepys_naval	117
pepys_financial	110

4.4 Context Distribution

Context	Chunks
Office	2,227
General	1,908
Home	1,215
Work	362
Social	305
Finance	229
Reflection	179
Emotion	140
Health	54
Family	28

5. Manifold Analysis: Measured Results

Embedding model: all-mpnet-base-v2 (768-dim, 110M params) **Corpus size for manifold analysis:** 8,413 entries **Hardware:** Apple Silicon CPU only

5.1 Intrinsic Dimensionality

The semantic structure of the Pepys corpus occupies far fewer dimensions than the 768-dim ambient space:

Metric	Value	Interpretation
TwoNN intrinsic dimensionality	14.26	The manifold is ~14-dimensional Effective PCA dimensions
Participation ratio	22.81	
PCA dims @ 90% variance	108	
PCA dims @ 95% variance	144	
PCA dims @ 99% variance	186	

The gap between TwoNN ID (14.26) and PCA ID (108) is characteristic of curved manifolds — the global PCA estimate overstates the local dimensionality. The diary’s se-

mantic content, despite being expressed in 768-dimensional space, lives on a roughly 14-dimensional surface.

5.2 MRL Retrieval Quality (MRR at k=5)

Matryoshka Representation Learning (MRL) analysis reveals how retrieval quality degrades as dimensionality is reduced:

Dimensions	MRR@5	Variance Explained
768 (full)	0.9615	94.2%
512	0.9615	95.8%
256	0.9038	98.9%
128	0.8654	100%
64	0.8558	100%

Key finding: **512-dim retains full retrieval quality** (MRR identical to 768-dim) while reducing vector storage by 33%. The corpus can be served at 512-dim with zero retrieval penalty.

6. Temporal Flight: A Novel Navigation Primitive

6.1 The Core Innovation

Standard embedding models discard temporal order. A Pepys entry from 1660 and one from 1669 may land near each other in semantic space if they discuss similar topics — the manifold is organized by meaning, not by time.

This pipeline introduces a novel primitive: **time as a navigable geometric axis**.

The temporal coordinate is appended to the embedding vector as a (D+1)-th dimension, scaled to match the mean L2 norm of the embedding vectors. The result is a **temporally-grounded embedding space** where time is literally a direction that a manifold navigator can face and fly.

```
def augment_with_time(embeddings, time_values, alpha=1.0):  
    # z-score normalize time values  
    t_norm = (time_values - time_values.mean()) / time_values.std()  
    # scale to match embedding magnitude  
    mean_norm = np.linalg.norm(embeddings, axis=1).mean()  
    t_scaled = t_norm * alpha * (mean_norm / sqrt(embeddings.shape[1]))  
    # append as (N+1)-th dimension  
    return np.column_stack([embeddings, t_scaled])
```

alpha controls the temporal weight: alpha=0 recovers the original embedding space; alpha=1 means time contributes as much as one typical embedding axis; alpha>1 makes time dominant.

6.2 Three Flight Modes

The TemporalFlyer builds a KNN graph ($k=10$) over the augmented (769-dim) space and supports three navigation modes:

Semantic flight: Navigate toward the semantically most-distant entry. Time is incidental — the navigator follows meaning.

Temporal flight: Orient heading along the time axis (`orient_in_time()`), then navigate forward in pure chronological order through the manifold.

Mixed flight: Blend semantic and temporal directions with a configurable `time_blend` parameter (0.0–1.0). The navigator spirals through both time and meaning simultaneously.

6.3 Flight Results (Full 8,413-entry corpus, 769-dim space)

Flight parameters: origin=1663-10-21, destination=1664-01-23, max_steps=150, alpha=1.0, $k=10$

Mode	Path Length	Monotonicity	Kendall τ	Mean Δt (years)	Total Span (years)
Semantic	79 steps	0.54	+0.19	1.56	2.01
Temporal	151 steps	0.52	−0.38	0.61	2.36
Mixed (blend=0.5)	142 steps	0.52	−0.15	1.13	2.78

Reading the results: - Semantic flight ($\tau = +0.19$): weakly forward in time — meaning-first navigation incidentally tends forward. - Temporal flight ($\tau = -0.38$): shorter time steps (0.61 yrs/step) — the navigator stays close in time at each hop. - Mixed flight: intermediate behavior — longer path, broader temporal coverage, moderate time coherence.

The negative Kendall τ values for temporal and mixed modes indicate the navigator is not simply traversing entries in timestamp order, but genuinely navigating a curved manifold where the temporal axis is orthogonal to some semantic axes. This is expected and desirable — it reveals the true geometry of the corpus.

6.4 The TurtleND Connection

The TemporalFlyer is built on TurtleND, an N-dimensional manifold navigator from the proteusPy library. Three new primitives were added to support temporal navigation:

- `expand_dim()` — grow the turtle’s basis set by one axis (enables $N \rightarrow N+1$ without rebuilding the navigator)
- `orient_in_time()` — rotate heading to face the temporal axis in augmented space

- `orient_toward(direction)` — rotate heading toward any arbitrary direction vector

These primitives generalize: any scalar quantity (sentiment, reading level, geographic distance, market volatility) can be appended as an extra navigable axis using the same `augment_with_time` pattern.

7. Novel Technical Contributions

7.1 Offline Semantic Pre-Computation at Ingest Time

The system pre-computes all semantic structure — topic labels, context categories, embeddings, graph edges — at ingest time using only pre-trained models (no fine-tuning, no LLM inference, no API calls). Query time requires only vector lookup and graph traversal. This inverts the standard cost model and enables query latency that is independent of corpus size.

7.2 Direct Temporal Database Writes (Bypassing LLM Extraction)

Temporal provenance is written directly to chunk frontmatter from parsed ISO timestamps in the source format. No LLM is involved in temporal grounding. This removes the single most common failure mode for local 4B-parameter models and makes the temporal dimension fully reliable regardless of model capability.

7.3 Configurable Temporal Dimension Augmentation

A scalar temporal coordinate is appended to the embedding vector, scaled to match the ambient embedding magnitude. The `alpha` parameter provides continuous control over the weight of time relative to semantic dimensions. This creates a unified $(D+1)$ -dimensional space where semantic and temporal navigation are interchangeable.

7.4 Multi-Modal Manifold Navigation with Time Primitives

Three navigation primitives (`expand_dim`, `orient_in_time`, `orient_toward`) extend an N -dimensional manifold navigator to operate in temporally-augmented spaces. Three navigation modes (semantic, temporal, mixed) allow continuous interpolation between meaning-first and time-first traversal of any corpus.

7.5 Parallel Embedding with Isolated Worker Processes

The embedder pipeline shards the corpus across `cpu_count()` worker processes, each loading an independent model instance. No shared memory, no GIL contention. Each shard is encoded independently; results are concatenated in original order. This achieves near-linear scaling with CPU count on encode-bound workloads.

7.6 Unified Orchestrator with Snapshot System

DiaryKG orchestrates the full pipeline — NLP transformation, corpus emission, DockKG indexing, and snapshot capture — through a single `build()` call. Point-in-time snapshots capture corpus metrics, graph counts, and temporal span, enabling version comparison across corpus rebuilds.

7.7 Hybrid Lexical + Semantic Search Without Inference

The resulting knowledge graph supports hybrid retrieval (BM25 + vector ANN) with no model loaded at query time. All embedding work is done during the build phase; the LanceDB index serves approximate nearest-neighbor queries directly from the pre-computed float32 vectors.

8. Performance Summary

All measurements on Apple Silicon laptop, CPU only, no GPU, no cloud services.

Task	Time	Scale
Full pipeline (ingest → KG)	~3 minutes	6,450 entries
Entries/second (preprocessing)	~100/sec	per core
Embedding (4 workers, batch=32)	included in above	6,647 chunks
Query latency	sub-millisecond	29,402 nodes
Inference at query time	none	—

9. Domain Generalization

The pipeline is domain-agnostic by design. The only domain-specific configuration is:

1. **Topics YAML** — list of topic labels and their keyword vocabularies
2. **Contexts YAML** — list of context categories
3. **Source format** — the `TIMESTAMP | TYPE | CATEGORY | CONTENT` format (the preprocessing layer that produces this is customizable)

Everything else — chunking, embedding, KG indexing, temporal augmentation, manifold navigation — is domain-independent.

Direct application targets with no code changes: - Personal journals and life logs
- Clinical notes and patient histories (temporal progression is critical) - Corporate meeting transcripts and decision logs - Legal case files and contract histories - Research literature corpora - Biographical and oral history archives - Any timestamped text corpus where temporal reasoning matters

10. System Status

As of 2026-03-26, the following components are working and validated:

Component	Status	Validated On
DiaryTransformer (5-phase NLP)	Working	Full Pepys corpus, 6,450 entries
DiaryKG.build() (full pipeline)	Working	Full corpus, ~3 min
DockKG SQLite index	Working	29,402 nodes, 355,250 edges
DockKG LanceDB vector index	Working	768-dim, 6,647 chunks
Semantic query (query_diary)	Working	Sub-millisecond
pepys_embedder.py (multi-process)	Working	8,413 entries, 4 workers
Manifold analysis (TwoNN, MRL-MRR)	Working	ID=14.26, MRR@768=0.9615
Temporal dimension augmentation	Working	769-dim, alpha=1.0
TemporalFlyer (3 flight modes)	Working	8,413 entries, k=10
orient_in_time() primitive	Working	Full corpus flight
Snapshot system	Working	Versioned captures

11. Implications

11.1 The Cost Model for Semantic Search Is Wrong

The prevailing assumption is that semantic search over large corpora requires ongoing inference costs. This system demonstrates that a 6,500-entry semantically-enriched corpus can be ingested once, in 3 minutes, on a laptop, with no ongoing inference cost whatsoever. The KG is a reusable, portable, diffable artifact.

11.2 Local Execution Is Now Viable for Temporally-Grounded Systems

The direct-temporal-write approach eliminates the LLM extraction failure mode that has blocked local execution for temporal memory systems. Any timestamp that can be parsed at ingest time can be written directly, with 100% reliability, regardless of model size.

11.3 Time Is a Navigable Dimension, Not Just a Filter

Standard corpus search treats time as a metadata filter (“show me entries from 1665”). This system treats time as a geometric dimension in embedding space — a direction

the navigator can face, fly toward, or blend with semantic directions. This enables qualitatively different queries: “take me through the diary following the thread of court politics as it evolves over time” is a navigation task, not a filter task.

11.4 Scalability Is Straightforward

The architecture scales by adding workers and disk. The 3-minute build time is dominated by embedding, which scales linearly with corpus size and inversely with worker count. A 10x larger corpus (65,000 entries) would require roughly 30 minutes at 4 workers, or 10 minutes at 12 workers. No architectural changes are required.

11.5 The Pattern Generalizes to Any Scalar Dimension

The temporal augmentation pattern — append a scaled scalar as an extra navigable dimension — works for any scalar: sentiment score, reading level, geographic coordinate, document age, author, topic cluster centroid. The navigation primitives generalize accordingly.

Appendix A: Key Files

File	Purpose
src/diary_kg/kg.py	DiaryKG orchestrator — full pipeline, build, query, snapshot
src/diary_kg/transformer.py	DiaryTransformer — 5-phase NLP, chunking, topic/context classification
benchmarks/pepys_embedder.py	Multi-process embedding pipeline → JSON cache
benchmarks/pepys_temporal_flight.py	Temporal augmentation + 3-mode manifold flight
benchmarks/pepys_mpnet_explorer.py	Manifold analysis: TwoNN, MRL-MRR, PCA intrinsic dimensionality
pepys/pepys_enriched_full.txt	Enriched corpus (3.3 MB, 23,235 lines)
.diarykg/graph.sqlite	Structural KG (109 MB)
.diarykg/lancedb/	Vector index (102 MB)
benchmarks/pepys_mpnet_results.json	Manifold analysis results
benchmarks/pepys_temporal_flight_results.json	Temporal flight results

Appendix B: Key Measured Numbers

Quantity	Value
Build time (full corpus)	~3 minutes
Corpus entries	6,450
KG nodes	29,402

Quantity	Value
KG edges	355,250
Embedding dimension	768
Augmented dimension (temporal)	769
TwoNN intrinsic dimensionality	14.26
MRR@768	0.9615
MRR@512 (same as @768)	0.9615
Topic categories	30+
Context categories	10
Query latency	sub-millisecond
Inference at query time	none